

지진데이터 처리 서버 모듈 구현을 위한

Earthworm 8.0 Docker 이미지 패키징 및 Docker Compose 배포 가이드

대상: Rocky Linux 9에서 빌드·검증 완료한 Earthworm 8.0 설치본

목적: 동일 환경을 다른 서버에 Docker 이미지 하나로 재현 가능하게 패키징 (Tank/아카이브/miniSEED 데이터 볼륨 포함)

방식: **docker compose build / up 기반 (docker run 단일 명령이 아닌 선언적 구성)**

1. 개요

Earthworm 자체는 공식 Docker 이미지를 제공하지 않습니다. 지금까지 Rocky Linux 9 서버에서 소스 빌드를 진행하며 실제로 겪은 문제들(CRB 저장소 비활성화, libtirpc-devel·gfortran 누락, -ltirpc 링크 누락, F77 legacy 인자 규칙 거부)뿐 아니라, 컨테이너를 실제로 기동하며 겪은 문제들(startstop이 요구하는 EW_INSTALLATION과 .desc 파일이 요구하는 EW_INST_ID가 서로 다른 변수라는 점, statmgr이 요구하는 SYS_NAME 누락, 백그라운드 실행으로 인한 interactive quit 미동작, 볼륨 디렉토리 UID 불일치, 비정상 종료 시 남는 lock 파일)까지 전부 Dockerfile·entrypoint.sh·docker-compose.yml 안에 반영해두면, 이후로는 docker compose build 한 번으로 동일한 환경이 재현됩니다.

이 문서는 이미지를 빌드하는 방법뿐 아니라, docker compose를 사용해 설정(params)과 로그를 컨테이너 바깥에 유지하면서 배포·운영하는 방법까지 다룹니다. 설정을 이미지 안에 굽지 않고 볼륨으로 분리해두면, 같은 이미지를 KMA 프로젝트용/콘소시엄 프로젝트용처럼 서로 다른 설정으로 재사용할 수 있습니다.

참고: 이 문서는 이전에 작성된 'Earthworm 다운로드 및 설치 가이드'에서 소스를 직접 빌드하며 확인한 STEP 1~STEP 4(설치 루트 생성 → 소스 다운로드 → 환경변수 → 컴파일) 내용을 Dockerfile 안으로 옮겨 담은 것입니다. 즉 서버에 맨손으로 설치하시던 절차가 이제 컨테이너 빌드 과정 안에서 자동으로 수행됩니다.

2. 사전 준비사항

항목	확인 방법
Docker Engine	docker --version (20.10 이상 권장)
Docker Compose	docker compose version (v2, docker-compose가 아니라 docker compose 하이픈 없는 서브커맨드)

빌드 서버 디스크 여유	이미지 레이어 + 빌드 캐시로 최소 3~4GB 여유 권장
네트워크	빌드 중 GitLab(gitlab.com) 및 Rocky Linux 패키지 저장소 접근 가능해야 함 — 에어갭 환경이면 4장 참고

Docker와 Compose 플러그인이 없다면 설치합니다.

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo systemctl enable --now docker
docker compose version      # 정상 설치 확인
```

참고: `sudo` 없이 `docker` 명령을 쓰고 싶으면 현재 계정을 `docker` 그룹에 추가하세요: `sudo usermod -aG docker $USER` (적용을 위해 재로그인 필요)

3. 설치 절차

STEP 1 작업 디렉토리 및 설정 파일 준비

Docker 이미지를 만들 별도 작업 공간을 만듭니다. 기존 `/opt/earthworm`과는 독립된 프로젝트 디렉토리입니다.

```
mkdir -p ~/earthworm-docker/run_config
cd ~/earthworm-docker
```

지금까지 `params` 디렉토리에서 직접 고친 설정 파일들(`.d`, `.desc`, `earthworm_commonvars.d` 등)을 그대로 이 프로젝트 안으로 복사해둡니다. 이 파일들은 이미지에 굽지 않고 볼륨으로 마운트할 것이므로, 여기 있는 원본이 곧 실제 운영 설정이 됩니다.

```
cp -r $EW_HOME/run/params ~/earthworm-docker/run_config/
mkdir -p ~/earthworm-docker/run_config/log
ls ~/earthworm-docker/run_config/params      # earthworm.d, statmgr.d, *.desc 등 확인
```

파형 등 대용량 데이터를 저장하는 모듈(`wave_serverv`의 Tank 파일, `trig2disk/waveman2disk`의 이벤트 아카이브 등)을 쓰실 계획이면, 로그·설정과는 별도로 데이터 전용 디렉토리도 미리 만들어둡니다. 이렇게 분리해두면 데이터만 별도 디스크·볼륨으로 옮기거나 백업 정책을 다르게 적용하기 쉽습니다.

```
mkdir -p ~/earthworm-docker/run_config/data/tanks      # wave_serverv Tank 파일
mkdir -p ~/earthworm-docker/run_config/data/archive    # trig2disk / waveman2disk 이벤트 아카이브
mkdir -p ~/earthworm-docker/run_config/data/mseed      # miniSEED 저장 모듈용 (예: ew2mseed 등)
```

참고: 최종 디렉토리 구조는 아래와 같습니다.

```
~/earthworm-docker/
```

```

├── Dockerfile
├── entrypoint.sh
├── docker-compose.yml
├── run_config/
│   ├── params/          # earthworm.d, statmgr.d, *.desc 등 설정 파일
│   ├── log/             # startstop 콘솔·모듈 로그
│   └── data/
│       ├── tanks/       # wave_server Tank 파일
│       ├── archive/     # trig2disk / waveman2disk 이벤트 아카이브
│       └── mseed/        # miniSEED 저장 모듈 출력 (예: ew2mseed)

```

STEP 2 Dockerfile 작성 (멀티스테이지 빌드)

~/earthworm-docker/Dockerfile 파일을 아래 내용으로 만듭니다. 1단계(builder)에서 소스를 내려받아 빌드하고, 2단계(runtime)에는 빌드 결과물과 실행에 필요한 런타임 라이브러리만 남겨 이미지 용량을 줄입니다.

```

# =====
# STAGE 1: 빌드 전용 (최종 이미지에는 포함되지 않음)
# =====
FROM rockylinux:9 AS builder
RUN dnf groupinstall -y "Development Tools" && \
    dnf install -y git make gcc gcc-c++ glibc-devel dnf-plugins-core && \
    dnf config-manager --set-enabled crb && \
    dnf install -y --allow-erasing libtirpc-devel gcc-gfortran libcurl-devel zlib-devel \
    ncurses-devel && \
    dnf clean all
ENV EW_HOME=/opt/earthworm
ENV EW_VERSION=earthworm_8.0
ENV PLATFORM=LINUX
# make unix 실행 전에 PATH가 잡혀 있어야 buildinfo 같은 빌드 중 생성되는 도구를 바로 찾아 씸
ENV PATH=${EW_HOME}/${EW_VERSION}/bin:${PATH}
WORKDIR ${EW_HOME}
RUN git clone https://gitlab.com/seismic-software/earthworm.git ${EW_VERSION}
WORKDIR ${EW_HOME}/${EW_VERSION}/src
# cc 래퍼: 모든 컴파일/링크 호출 맨 끝에 -ltirpc 를 강제로 추가
RUN printf '#!/bin/sh\nexec /usr/bin/cc "$@" -ltirpc\n' > /usr/local/bin/cc && \
    chmod +x /usr/local/bin/cc
# f77 래퍼: hyp2000 등 레거시 FORTRAN 77 모듈용.
# 최신 gfortran(10+)은 옛날 F77 코드의 느슨한 인자 규칙을 기본 거부하므로
# -fallow-argument-mismatch -std=legacy 로 완화해줌
RUN printf '#!/bin/sh\nexec /usr/bin/gfortran -fallow-argument-mismatch -std=legacy -w \
"$@"\n' > /usr/local/bin/f77 && \
    chmod +x /usr/local/bin/f77
RUN export GLOBALFLAGS="-D_LINUX -Dlinux -D_INTEL -D_USE_SCHED -D_USE_PTHREADS - \
D_USE_TERMIO -D_FILE_OFFSET_BITS=64 -I${EW_HOME}/${EW_VERSION}/include - \
I/usr/include/tirpc" && \
    export EW_BITS=64 && \
    export QLIB_BITS=64 && \
    export USE_CC_BITS=true && \
    make unix 2>&1 | tee /tmp/build.log && \
    ! grep -q "^make.*Error [0-9]" /tmp/build.log
# =====
# STAGE 2: 실제 배포용 (런타임 라이브러리만 포함, 가볍게 유지)
# =====
FROM rockylinux:9 AS runtime
# 컴파일용 -devel 패키지가 아니라 '실행에 필요한' 런타임 라이브러리만 설치
# tini는 EPEL 저장소에 있어 BaseOS/AppStream만으로는 설치되지 않으므로 epel-release를 먼저 활성화

```

```

RUN dnf install -y epel-release && \
    dnf install -y libtirpc libgfortran zlib ncurses procps-ng tini && \
    dnf clean all
ENV EW_HOME=/opt/earthworm
ENV EW_VERSION=earthworm_8.0
ENV EW_PARAMS=${EW_HOME}/run/params
ENV EW_LOG=${EW_HOME}/run/log
# EW_INSTALLATION: startstop(GetLocalInst)이 설치 ID를 조회할 때 읽는 변수명
# EW_INST_ID: statmgr.desc 등 .desc 파일 안의 ${EW_INST_ID} 치환에 쓰이는 변수명 (용도가 다르므로 둘 다 필요)
ENV EW_INSTALLATION=INST_UNKNOWN
ENV EW_INST_ID=INST_UNKNOWN
# statmgr이 로컬 시스템 이름으로 사용 (statmgr.d에 system 라인이 없을 때 대체값)
ENV SYS_NAME=earthworm-docker
ENV PATH=${PATH}:${EW_HOME}/${EW_VERSION}/bin
# 1단계에서 빌드된 결과물만 복사 (소스 빌드 도구는 최종 이미지에 없음)
COPY --from=builder ${EW_HOME}/${EW_VERSION} ${EW_HOME}/${EW_VERSION}
# run 디렉토리는 비워둠 - 실제 설정·데이터는 docker-compose.yml의 볼륨으로 마운트
RUN mkdir -p ${EW_HOME}/run/params ${EW_HOME}/run/log \
    ${EW_HOME}/run/data/tanks ${EW_HOME}/run/data/archive ${EW_HOME}/run/data/mseed
# 비root 계정으로 실행 (보안 권장사항). UID를 1000으로 고정해 베이스 이미지가 바뀌어도
# 항상 동일한 UID를 갖도록 함 (STEP 6의 호스트 디렉토리 UID 맞추기와 짝이 되는 값)
RUN useradd -m -u 1000 -d /home/eqm eqm && \
    chown -R eqm:eqm ${EW_HOME}
USER eqm
COPY --chown=eqm:eqm entrypoint.sh /usr/local/bin/entrypoint.sh
RUN chmod +x /usr/local/bin/entrypoint.sh
# tini를 PID 1로 사용 - SIGTERM을 자식(startstop)에 전달하고 좀비 프로세스를 정리해줌
ENTRYPOINT ["/usr/bin/tini", "--", "/usr/local/bin/entrypoint.sh"]

```

참고: GLOBALFLAGS export를 빠뜨리면 STAGE 1(builder)의 make unix가 diagnostic_tools/buildinfo까지는 성공하고, 이후 libsrc 컴파일 단계에서 "earthworm.h: No such file or directory"(예: libsrc/earlybird/filters.c)로 실패합니다. 물리 서버 설치 때는 environment/ew_linux.bash를 source해서 GLOBALFLAGS(-I\${EW_HOME}/\${EW_VERSION}/include 포함)를 잡아준 뒤 빌드했는데, Dockerfile에는 이 단계가 없어서 생기는 문제입니다.

⚠ 설정 파일을 이미지에 굶지 않는 이유 이전 안내에서는 run_config/params를 COPY로 이미지 안에 직접 넣었지만, 이번 버전은 볼륨 마운트 방식으로 바꿨습니다. 설정을 이미지에 구우면 서버(IP)나 Installation ID가 다른 환경마다 이미지를 다시 빌드해야 합니다. 볼륨으로 분리하면 이미지는 한 번만 빌드하고, 배포 대상마다 run_config 폴더 내용만 바뀌서 재사용할 수 있습니다.

STEP 3 entrypt.sh 작성

컨테이너가 시작될 때 실행될 스크립트입니다. Docker 컨테이너는 반드시 '포그라운드에서 도는 프로세스 하나'가 있어야 살아있는 상태로 유지되므로, 예전에 만든 ew_start처럼 백그라운드로 돌리는 게 아니라 startstop을 포그라운드(exec)로 그대로 실행합니다.

```

#!/bin/bash
set -e

# 볼륨 마운트가 제대로 되어 설정 파일이 존재하는지 확인
if [ ! -f "${EW_PARAMS}/startstop_unix.d" ]; then
    echo "[오류] ${EW_PARAMS}/startstop_unix.d 를 찾을 수 없습니다."
    echo "docker-compose.yml의 volumes 설정에서 run_config/params 마운트를 확인하세요."

```

```

    exit 1
fi

# 이전 컨테이너가 비정상 종료(강제 kill, OOM 등)되며 남긴 stale lock 제거
rm -f "${EW_LOG}/startstop_unix.d.lock"

cd "${EW_PARAMS}"

# startstop을 포그라운드로 직접 실행 (PID 1은 ENTRYPOINT의 tini).
# 백그라운드(&)로 띄우면 interactive 입력(quit, restart 등)이 tty에서
# 제대로 전달되지 않아 fgets()가 EAGAIN으로 실패하는 문제가 있었음
exec startstop

```

참고: 컨테이너는 자체 IPC 네임스페이스를 갖기 때문에, 예전 bare-metal 환경에서 FIFO로 quit<cr>을 보내던 것과 같은 정교한 처리가 필요 없습니다. 이전 버전에서는 entryptoint.sh 안에서 bash가 startstop을 백그라운드(&)로 띄우고 trap으로 SIGTERM을 수동 전달하는 방식을 썼는데, 이 경우 startstop이 tty를 포그라운드로 붙잡지 못해 인터랙티브 quit 입력이 씹히는 문제가 있었습니다(fgets EAGAIN). 그래서 이번 버전은 ENTRYPOINT에서 tini를 PID 1로 두고 entryptoint.sh는 startstop을 exec로 그대로 포그라운드 실행합니다 — tini가 SIGTERM 전달과 좀비 프로세스 정리를 대신 해주므로 entryptoint.sh에서 trap을 직접 짤 필요가 없어졌고, startstop이 tty를 정상적으로 붙잡아 quit이 즉시 반영됩니다. docker stop이 보내는 SIGTERM을 startstop에 그대로 전달하고, 컨테이너가 종료되면 그 안의 공유메모리·세마포어는 컨테이너와 함께 자동으로 정리됩니다.

STEP 4 docker-compose.yml 작성

~/earthworm-docker/docker-compose.yml 파일을 아래와 같이 만듭니다. 포트 번호는 실제 slink2ew.d, wave_serverv.d, import_generic.d, export_generic.d에 설정된 값으로 반드시 바꿔주세요.

```

services:
  earthworm:
    build:
      context: .
      dockerfile: Dockerfile
    image: kitvalley/earthworm:8.0
    container_name: earthworm-8
    restart: unless-stopped

    environment:
      - EW_INSTALLATION=INST_UNKNOWN    # startstop이 조회하는 설치 ID
      - EW_INST_ID=INST_UNKNOWN         # .desc 파일 내 ${EW_INST_ID} 치환용 (별도 용도)
      - SYS_NAME=earthworm-docker      # statmgr이 사용하는 로컬 시스템 이름

    volumes:
      - ./run_config/params:/opt/earthworm/run/params
      - ./run_config/log:/opt/earthworm/run/log
      # 파형 등 대용량 데이터 – wave_serverv Tank 파일, trig2disk/waveman2disk 아카이브,
      miniSEED
      - ./run_config/data/tanks:/opt/earthworm/run/data/tanks
      - ./run_config/data/archive:/opt/earthworm/run/data/archive
      - ./run_config/data/mseed:/opt/earthworm/run/data/mseed

    ports:
      # 예시입니다. wave_serverv.d 등에 설정된 실제 포트에 교체하세요.
      - "16022:16022" # wave_serverv 조회 포트 (예시)
      # - "18000:18000" # slink2ew가 접속하는 원격 SeedLink 포트는 아웃바운드라 노출 불필요

```

```
# 원격 SeedLink 서버가 특정 사설 대역에서만 접근 가능하다면
# host 네트워크 모드가 더 간단할 수 있습니다 (Linux 전용):
# network_mode: host
```

```
healthcheck:
  test: ["CMD", "pgrep", "-x", "startstop"]
  interval: 30s
  timeout: 5s
  retries: 3
```

참고: `network_mode: host`를 쓰면 `ports` 항목은 무시되고 컨테이너가 호스트 네트워크를 그대로 씁니다. `slink2ew`가 접속해야 하는 원격 서버가 사내망 특정 라우팅 경로에만 있다면 이 방식이 더 간단할 수 있습니다. 반대로 여러 컨테이너를 격리해서 운용하고 싶다면 기본 브리지 네트워크 + `ports` 매핑 방식을 유지하세요.

△ 모듈 설정 파일의 저장 경로도 컨테이너 내부 경로로 맞춰야 함 볼륨을 연결하는 것만으로는 부족합니다. `wave_serverV.d`의 Tank 지시자, `trig2disk.d/waveman2disk.d`의 아카이브 디렉토리 지시자가 여전히 예전 bare-metal 경로(예: `/opt/earthworm/run/tanks`)를 가리키고 있다면, 컨테이너 안에서는 그 경로가 존재하지 않아 오류가 납니다. 아래처럼 컨테이너 내부 마운트 경로(`/opt/earthworm/run/data/tanks`, `/opt/earthworm/run/data/archive`)로 수정하거나, 반대로 `docker-compose.yml` 쪽의 마운트 대상 경로를 기존 `.d` 파일이 참조하는 경로에 맞춰주세요(`.d` 파일을 아예 건드리지 않아도 되는 더 안전한 방법).

```
# 예시 - wave_serverV.d (.d 파일을 그대로 두고 싶다면, 이 값은 손대지 않음)
Tank      /opt/earthworm/run/data/waveserver/WAVE_RING.tnk 1000
TankStructFile /opt/earthworm/run/data/waveserver/p1000-1.str

# 대신 docker-compose.yml에서 컨테이너 내부 경로를 기존 값에 맞춤
# - ./run_config/data/tanks:/opt/earthworm/run/data/waveserver

# 예시 - trig2disk.d / waveman2disk.d
OutDir    /opt/earthworm/run/data/archive

# 예시 - miniSEED 저장 모듈(ew2mseed 등, 실제 키워드는 해당 .d 파일에서 확인)
OutDir     /opt/earthworm/run/data/mseed
# 또는
MSEEDDir   /opt/earthworm/run/data/mseed
```

실제 지시자 이름과 문법은 사용 중인 모듈 버전에 따라 다를 수 있으니, `run_config/params` 안의 해당 `.d` 파일을 열어 현재 어떤 키워드로 경로를 지정하고 있는지 먼저 확인한 뒤, `.d` 파일을 바꿀지 `docker-compose.yml`의 마운트 경로를 바꿀지 정하시면 됩니다. 이미 운영 중이던 `.d` 파일이라 Tank 인이 여러 줄이라면, `.d` 파일은 그대로 두고 `docker-compose.yml` 쪽 경로만 맞추는 쪽이 오타 위험이 적어 더 안전합니다.

```
grep -iE "outdir|mseed|directory" $EW_HOME/run/params/<해당모듈>.d
```

데이터 볼륨을 얼마나 크게 잡을지는 채널 수, 샘플링 레이트, 보관 기간으로 결정됩니다. 디스크 공간이 부족해지면 `wave_serverV`가 오래된 Tank 데이터를 덮어쓰기 시작하므로, 운영 전 여유 있게 산정하시고 `df -h`로 주기적으로 확인하시는 것을 권장합니다.

참고: 데이터 양이 많아 호스트 파일시스템 성능이 병목이 되는 경우, 위 `bind mount(/run_config/data/...)` 대신 Docker 명명된 볼륨(named volume)을 쓰면 스토리지 드라이버가 최적화를 더 잘 해주는 경우가 있습니다. 다만 이 경우 데이터가 docker volume 내부 관리 영역에 들어가므로 호스트에서 직접 파일을 들여다보기가 조금 불편해집니다. 지금 단계에서는 확인·백업이 쉬운 `bind mount` 방식을 권장합니다.

STEP 5 이미지 빌드

```
cd ~/earthworm-docker
docker compose build
```

빌드 로그 안에서 STAGE 1(builder)의 `make unix` 진행 상황이 그대로 출력됩니다. 처음 빌드할 때는 소스 다운로드 + 전체 컴파일 때문에 수 분 정도 걸릴 수 있습니다. 완료 후 이미지가 생성됐는지 확인합니다.

```
docker images | grep earthworm
```

STEP 6 볼륨 디렉토리 소유권(UID) 맞추기

이미지가 빌드된 지금 시점에야 비로소 컨테이너 안 `eqm` 계정의 UID를 확인할 수 있습니다 — 그래서 이 작업은 STEP 4가 아니라 이미지 빌드가 끝난 지금(STEP 5 다음) 진행합니다.

△ 볼륨 마운트 시 디렉토리 소유권(UID) 불일치 문제 `docker compose up`은 호스트에 없는 볼륨 디렉토리를 자동으로 만들어주지만, 이때 소유자가 `root`가 됩니다. 컨테이너는 `Dockerfile`에서 만든 `biroot` 계정(`eqm`)으로 실행되므로, `root` 소유 디렉토리에는 쓰기 권한이 없어 `mseed-tank` 파일 저장이 `Permission denied`로 실패할 수 있습니다. 반드시 호스트에서 디렉토리를 먼저 만들고, 컨테이너 내부 `eqm` 계정과 동일한 UID로 소유권을 맞춰주세요.

```
# 1) 방금 빌드한 이미지로 컨테이너 안 eqm의 UID 확인
docker compose run --rm earthworm id eqm
# 예: uid=1000(eqm) gid=1000(eqm)

# 2) 호스트 디렉토리 소유자를 동일 UID로 맞춤
sudo chown -R 1000:1000 ~/earthworm-docker/run_config/data/mseed
sudo chown -R 1000:1000 ~/earthworm-docker/run_config/data/tanks
sudo chown -R 1000:1000 ~/earthworm-docker/run_config/data/archive
```

참고: 여기서 확인된 UID가 1000이 아니라면 위 `chown` 명령의 숫자를 실제 값으로 바꿔주세요. 이 작업은 컨테이너를 처음 기동하기 전에 마쳐야 하며, 이후 데이터 디렉토리를 추가할 때마다 동일하게 적용해야 합니다.

STEP 7 컨테이너 기동

△ 먼저 테스트용 컨테이너를 종료했는지 확인 STEP 5~6 사이에 동작 확인용으로 `docker compose run --rm earthworm` 등을 실행해보셨다면, 그 컨테이너를 먼저 정상 종료(인터랙티브 프롬프트에 `quit` 입력, 또는 `docker compose down`)한 뒤에 STEP 7을 진행하세요. `run_config/log` 볼륨을 공유하기 때문에, 테스트 컨테이너가 `startstop_unix.d.lock`을 아직 쥐고 있는 상태에서 `up -d`를 실행하면 `"lock already locked by`

another instance" 오류가 납니다.

```
docker compose up -d
docker compose ps          # 상태 확인 (Up, healthy 여야 정상)
```

참고: STEP 4에서 `docker-compose.yml`을 수정했거나 STEP 6에서 디렉토리 권한을 나중에 바꾼 경우에는 `docker compose up -d --force-recreate` 로 컨테이너를 재생성해야 반영됩니다. 단순 `docker compose restart`로는 볼륨 구성 변경이 적용되지 않습니다. 또한 컨테이너를 처음 기동할 때 로그에 "(MOD_STARTSTOP) module dead"가 한 번 찍힐 수 있는데, 이는 `statmgr`이 `startstop`의 첫 하트비트를 컨테이너 초기 기동 지연 때문에 놓친 것으로 이후 반복되지 않으면 무시해도 되는 일회성 현상입니다. `docker compose logs`에서 같은 메시지가 반복해서 나타난다면 `statmgr.d`가 참조하는 `descriptor` 파일의 `tsec`(하트비트 타임아웃) 값을 늘려주세요.

STEP 8 상태 확인 및 로그

```
docker compose logs -f earthworm    # 실시간 로그 (Ctrl+C로 빠져나와도 컨테이너는 계속 실행)
docker compose exec earthworm bash  # 컨테이너 내부 셸 진입
```

컨테이너 내부에서는 기존과 동일한 진단 도구를 그대로 사용할 수 있습니다.

```
# 컨테이너 내부 셸에서
sniffing WAVE_RING
sniffwave STA COMP NET LOC
```

STEP 9 중지 / 재시작

명령	동작
<code>docker compose stop</code>	컨테이너 정지 (SIGTERM → <code>entrypoint.sh</code> 가 <code>startstop</code> 에 전달 → 정상 종료). 컨테이너/설정은 남아있음
<code>docker compose start</code>	정지된 컨테이너 재시작
<code>docker compose restart</code>	stop + start 한 번에
<code>docker compose down</code>	컨테이너 삭제 (볼륨의 <code>params/log</code> 는 호스트에 남아있으므로 설정 유실 없음)
<code>docker compose up -d --build</code>	Dockerfile 변경 후 재빌드하며 재기동

4. 폐쇄망(에어갭) 서버로 배포하기

빌드 서버에서는 인터넷으로 GitLab·패키지 저장소에 접근할 수 있지만, 실제 운영 서버는 폐쇄망(인터넷 차단)인 경우가 많습니다. 이전에 Tomcat 컨테이너를 폐쇄망에 배포하실 때 쓰신 것과 동일한 `docker save` / `docker load` 방식을 그대로 적용할 수 있습니다.

STEP A 빌드 서버에서 이미지 저장

```
docker save kitvalley/earthworm:8.0 | gzip > earthworm_8.0_image.tar.gz
ls -lh earthworm_8.0_image.tar.gz
```

STEP B 폐쇄망 서버로 전달

USB, 내부 파일 전송 서버 등 반출입 절차에 따라 아래 파일들을 함께 옮깁니다.

- earthworm_8.0_image.tar.gz (이미지 파일)
- docker-compose.yml
- run_config/ 전체 디렉토리 (params, log)

STEP C 폐쇄망 서버에서 이미지 로드 및 기동

```
gunzip -c earthworm_8.0_image.tar.gz | docker load
docker images | grep earthworm # 로드 확인

cd ~/earthworm-docker # docker-compose.yml, run_config가 있는 위치
docker compose up -d
```

참고: docker-compose.yml의 build 항목이 있어도, image 항목에 지정된 이미지가 이미 로드되어 있으면 docker compose up은 재빌드하지 않고 기존 이미지를 그대로 사용합니다. 폐쇄망에서는 build 관련 지시문이 있어도 문제되지 않지만, 혹시 실수로 재빌드가 시도되지 않도록 폐쇄망용 compose 파일에서는 build 섹션을 주석 처리해두는 것도 안전한 방법입니다.

5. bare-metal 백그라운드 실행과의 차이

항목	bare-metal (ew_start/ew_stop)	Docker Compose
백그라운드 유지 방식	FIFO + nohup + keepalive 프로세스로 직접 구현	컨테이너 자체가 백그라운드 단위 (docker compose up -d)
정상 종료 신호 전달	FIFO에 quit 문자열을 echo	docker stop이 보내는 SIGTERM을 entrypoint.sh가 startstop에 전달
공유메모리/세마포어 정리	수동 관리 필요 (여러 인스턴스 충돌 주의)	컨테이너 고유 IPC 네임스페이스라 컨테이너 종료 시 자동 정리
여러 인스턴스 운용	run_kma, run_pipeline처럼 디렉토리 분리 + 함수 인자화 필요	docker-compose.yml에 서비스 (service)를 추가하는 것만으로 격리된 인스턴스 구성 가능

참고: 두 방식을 굳이 양자택일할 필요는 없습니다. 개발 중인 서버 모듈을 빠르게 반복 테스트할 때는 bare-metal + ew_start/ew_stop이 더 가볍고, 여러 서버에 동일 환경을 배포하거나 KMA/컨소시엄 프로젝트처럼 다중 인스턴스를 운영할 때는 Docker Compose 쪽이 관리가 쉽습니다.

6. 참고 자료

- Earthworm GitLab 저장소: <https://gitlab.com/seismic-software/earthworm>
- Earthworm Central (공식 문서 허브): <http://www.earthwormcentral.org/>
- Docker Compose 공식 문서: <https://docs.docker.com/compose/>

참고: Earthworm 프로젝트 자체는 공식 Docker 이미지를 배포하지 않습니다. 본 문서의 Dockerfile은 2026년 8월 Rocky Linux 9 환경에서의 실제 빌드·기동 검증을 기반으로 자체 작성한 것입니다.